



Java Learning Center

No.1 In Java Training & placement

DSA

Module 6

Arrays

Author

Srinivas Dande



My **JAVA
LEARNING
CENTER**

6. Arrays

- ♦ Array is a collection of elements which are of same data type.
- ♦ Array is also called as Homogeneous data structure.
- ♦ Elements of an array will be stored in contiguous memory locations.
- ♦ Arrays are objects in Java.
- ♦ Three tasks to remember while you are working with arrays:
 - Array Declaration
 - Array Construction
 - Array Initialization
- ♦ Arrays can be constructed with multiple dimensions i.e 1-D Array, 2-D Array etc.

6.1. Single Dimensional Arrays

- ♦ Single Dimensional array can be called as 1-D Array.

6.1.1. Array Declaration and Construction

- ♦ You can declare and construct an array in one statement also.

Syntax:

```
<dataType> <refVarName>[] = new <dataType>[<size>];
<dataType> []<refVarName> = new <dataType>[<size>];
```

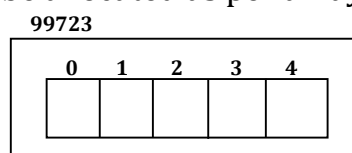
Ex:

```
int arr[] = new int[5];
int []arr = new int[5];
```

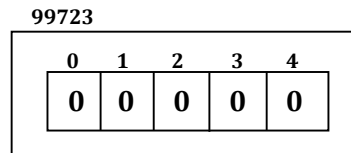
- 1) Allocates 8 bytes of memory for the reference variable and initializes with null value.



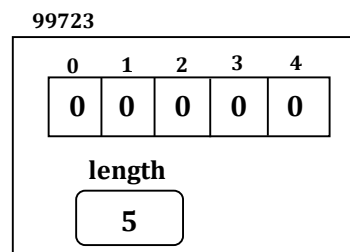
- 2) Memory blocks will be allocated as per array size.



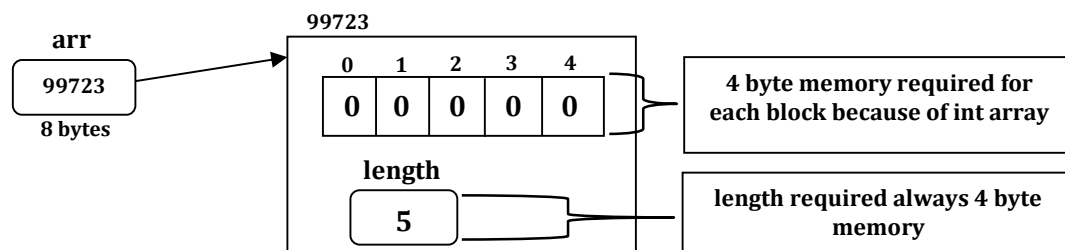
3) Memory blocks will be initialized with default value as per array data type.



4) Memory will be allocated for length variable and will be initialized with size of an array.

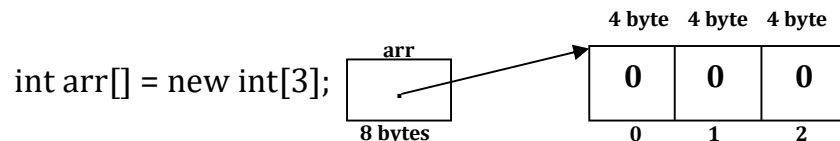


5) Array object address will be assigned to reference variable.



6.1.2. Array Initialization

- Once array is constructed then array elements will be initialized with default values as per array data type.
- You can initialize array elements with your own value using index representation.



- Here we are creating an array with size 3, so only three elements will be stored in this array.

`arr[0]` -> Represents the first element which contains value 0.

`arr[1]` -> Represents the second element which contains value 0.

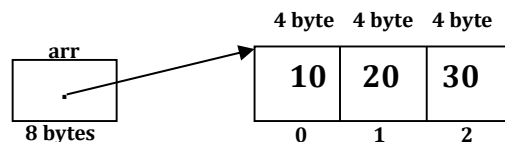
`arr[2]` -> Represents the third element which contains value 0.

- You can initialize the array elements as follows:

`arr[0] = 10;`

`arr[1] = 20;`

`arr[2] = 30;`



- Accessing the elements after initialization

`arr[0]` -> Represents the first element which contains value 10.

`arr[1]` -> Represents the second element which contains value 20.

`arr[2]` -> Represents the third element which contains value 30.



Lab1.java

```
package com.jlccindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab1 {
    public static void main(String[] args) {

        // 1. Create Array
        int arr[] = new int[5];

        // 2.find the Length of Array
        int n = arr.length;
        System.out.println(n);

        // 3.Access Elements of Array
        for (int i = 0; i < n; i++)
            System.out.println(arr[i]);

        // 4. Initialize the Array
        arr[0] = 11;
        arr[2] = 33;
        arr[4] = 55;
        System.out.println("-----");

        // Access Elements of Array
        for (int i = 0; i < n; i++)
            System.out.println(arr[i]);
    }
}
```



6.1.3 Array Declaration, Construction and Initialization

- ♦ You can declare, construct and initialize an array in one statement also.

Syntax:

```
<dataType> <refVarName>[] = {<val1>,<val2>,<val3>,...,<valN>};
```

Ex:

```
int arr[]={10,20,30};  
String names[]{"Hello","Srinivas","Dande"};
```

Lab2.java

```
package com.jlcindia.arrays;  
  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
  
public class Lab2{  
    public static void main(String[] args) {  
  
        int arr[] = {10,20,30,40,50};  
  
        int n= arr.length;  
  
        // Access Elements of Array  
        for (int i = 0; i < n; i++)  
            System.out.println(arr[i]);  
    }  
}
```

6.1.4. Why and Why Not Arrays?

- ♦ **Limitations with array:**

- ✓ Array is a homogeneous data structure i.e only one type of elements can be stored in the Array.
- ✓ Array is static in nature i.e size of an array cannot be modified.
- ✓ Insert and delete operations require more shifting of elements.
- ✓ You need to write the Code for doing any Operations on Arrays. No Built-In Funcatioality provided in Arrays.

- ♦ **Advantages with array:**

- ✓ Arrays are faster in Accessing the Elements.
- ✓ Arrays are Cache-Friendly



6.1.5 Dynamic Size Array - ArrayList

- ♦ ArrayList is implemented as a resizable array.
- ♦ ArrayList elements will be stored internally using indexing notation.
- ♦ This is one of the most widely used concrete class.
- ♦ It is fast to access the elements, but slow to insert and delete the elements.
- ♦ Size of ArrayList will be growable dynamically.
- ♦ Default Capacity is 10. i.e When You create ArrayList, It allocates the Space for 10 Elements in advance.
- ♦ When you add elements then size will be increased.
- ♦ Following the Formula for Capacity Calculation.

In Java 6

$$\text{newCapacity} = (\text{oldCapacity} * 3) / 2 + 1;$$

In Java 7

$$\text{newCapacity} = \text{oldCapacity} + (\text{oldCapacity} >> 1)$$

- ♦ ArrayList comes with many built-methods as follows.
 - `int size()`
 - `boolean isEmpty()`
 - `Object get(int idx)`
 - `boolean add(Object obj)`
 - `void add(int idx, Object obj)`
 - `boolean remove(Object obj)`
 - `Object remove(int idx)`
 - `Object set(int idx, Object obj)`
 - `boolean contains(Object obj)`



Lab3.java

```
package com.jlcindia.arrays;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

import java.util.ArrayList;
import java.util.List;

public class Lab3{
    public static void main(String[] args) {

        List<Integer> mylist = new ArrayList<>();

        mylist.add(10);
        mylist.add(20);
        mylist.add(30);
        mylist.add(40);

        System.out.println(mylist);
        mylist.add(50);
        System.out.println(mylist);
        mylist.remove(Integer.valueOf(40));
        System.out.println(mylist);
        //ArrayList has many Built-In methods to Use
    }
}
```



6.2. Operations on Arrays

Lab4.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Search for given Element in the Unsorted Array

public class Lab4 {
    static int linearSearch(int arr[], int element) {
        int n = arr.length;

        for (int i = 0; i < n; i++) {
            if (arr[i] == element) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int arr[] = { 25, 10, 5, 15, 30, 20, 50 };
        int element = 25;

        int index = linearSearch(arr, element);

        System.out.println(index);
        if (index == -1)
            System.out.println("Element Not Found");
        else
            System.out.println("Element Found at Index : " + index);
    }
}

// Time Complexity - - L.S O(n)
// Aux Space - O(1)
```